DOCUMENTO

Día 2

Apuntes del curso — Formación Copilot IA

Generado automáticamente desde `dia2.md`

IA

Copilot es una IA? NO

Qué es Copilot? Es una palabra que agrupa MUCHAS HERRAMIENTAS/SERVICIOS que tienen que ver con IA en Microsoft. La familia de MS Copilot incluye herramientas/servicios como GitHub Copilot, Copilot Chat, y otras integraciones de IA en productos de Microsoft.

Realmente que es la IA? Programa que despues de entrarse realiza una tarea simulando razonamientos humanos. A día de hoy ese término lo aplicamos casí en exclusiva a MODELOS GENERATIVOS, que son modelos entrenados para generar contenido nuevo a partir de datos de entrenamiento.

Los más famosos/usados son los LLM (Large Language Models), que son modelos de lenguaje de gran tamaño entrenados con enormes cantidades de texto para generar respuestas TEXTUALES coherentes y contextualmente relevantes.

Yo les paso un TEXTO y ellos devuelven otro TEXTO.

Pero hay muchos:

- Modelos de generación de video:
  - Modelos de texto a video
  - Modelos de imagen a video
- Modelos de generación de imagen:
  - Modelos de texto a imagen
  - Modelos de imagen a imagen
- Modelos de generación de audio:
  - Modelos de texto a audio

    - Modelos de imagen a audio
- Modelos que generan texto desde audios (transcribir)
- Modelos que generan texto desde una imagen/video (describir)

Dentro de los LLMs (modelos de texto) algunos famosos:

- OpenAI GPT
- Anthropic Claude, Fable, Mythos
- Google Gemini
- X Grok
- Meta (Facebook) LLaMA

Dentro de cada familia hay: Modelos más o menos grandes y que van evolucionando en distintas versiones a lo largo del tiempo.

```
CODE

CLAUDE:
    pequeños: Haiku     4.5 4.6 5.0
    medianos: Sonnet    4.5 4.6 5.0
    grandes:  Opus      4.5 4.6 5.0

Fable 5.2 (recién salido) Ahora mismo es el modelo más potente que existe en el mundo

Ha superado al anteriormente más listo: Google Gemini 4. Que salió hace 2 semanas...

Este superó GPT 6.1 Astra (que le duró un mes el reino.)
```

Nosotros hablamos/escribimos directamente con estos modelos? NO. Lo hacemos a través de herramienats...normalmente de tipo chat.. aunque no solo.

Ejemplos:

- GitHub Copilot (integrado en editores de código)
- Copilot Chat (integrado en productos de Microsoft)
- Openclaw
- Otras aplicaciones que usan LLMs para generar contenido a partir de texto, imagen, audio, etc.

Estas herramientas han evolucionado para facilitar la interacción con los modelos de IA. Antiguamente la respuesta que el modelo ofrecia a una pregunta mía, estas herramientas me la devolvían directamente.

Pero esto cambió hace tiempo ya (más de un año).

Hoy en día estamos en lo que llamamos la ERA AGENTICA!

Agente de IA

```
CODE

NOSOTROS                        AGENTE DE IA / ENTORNO DE EJECUCION

                                chat            modelo          otros programas
-------------------------       ----------------------------------------------
mensaje/pregunta/tarea     ---> github copilot + claude opus + terminal donde escribi
                                                             + navegador de internet
                                                             + intérprete de python
                                                             + otras herramientas ext
Y el modo de funcionamiento cambia.
- Yo mando mensaje/tarea al agente de IA
- La herramienta de chat toma el mensaje y lo manda al modelo
- El modelo genera una respuesta, que puede contener:
  - texto
  - programas / comandos que quieran ejecutar
- En caso que se manden progrmas por parte del modelo, el chat los ejecuta en el ento
  y devuelve el resultado al modelo.
- El modelo decide con ese resultado que hacer después, generando una nueva respuesta
- Cuando el modelo considera que ha completado la tarea, genera una respuesta final
- Que la herramienta de chat nos enseña.
```

En este modelo, hay un cambio ENORME de planteamiento. No es que la herramienta (agente/IA) me consteste... es que HACE COSAS por mí. COSAS que pueden tener impacto en el mundo real.

Esto es peligroso. No ya porque vaya a aniquilar la humanidad.

- Sino porque me puede borrar el disco duro de la computadora.. y dejarme sin archivos.
- O borrarme todos mis emails
- O mandar un email comprometido con información sensible a alguien sin que yo lo autorice.
  ...

Cuando trabajamos de esta forma (con agentes) hay que tener mucho cuidado!

Lo más importante establecer muchos límites a la acción del agente de IA:

- Limitando FISICAMENTE el entorno de trabajo disponible
- Dando reglas MUY CLARAS de cosas que puede hacer y de cosas QUE NO DEBE HACER BAJO NINGUN CONCEPTO!

Pero... cómo damos tanta regla? En un prompt no escribimos tanto? CONTEXTO

Qué es el contexto?

El contexto es el TEXTO que recibe el modelo de IA cada vez que el chat le manda algo (un mensaje mio o la salida de una herramienta). Las IAs (los modelos, especialmente los LLM) lo que reciben es un TEXTO y producen otro TEXTO. El texto que receiben es el CONTEXTO. El prompt que yo escribo no es directamente lo que recibe la IA. El prompt pasa a formar parte del contexto!

El chat, al generar el CONTEXTO incluye al final de todo ese contexto el PROMPT que yo he pasado. PEro el prompt suelerepresnetar menos de un 1%..incluso de un 0.1% del contexto total.

El otro día mandé un mensaje de 7 palabras... y en contexto había 21k palabras.

7/21000 ≈ 0.033%

```
CODE

YO(Humano) --> prompt --> CHAT --> CONTEXTO                --> MODELO
                                    (incluye el prompt )
```

Lo que pasa es que el contexto incluye muchas más cosas, además del prompt:

- Prompts anteriores + respuestas anteriores del modelo (HISTORIAL DE LA CONVERSACION)
- Instrucciones / límites que ponemos al agente (aque se crean mediante archivos de texto)
- Información del entorno y de las herramienats que el modelo tiene disponibles
- Información de servidores MCP disponibles.
- Información acerca de mi, mis preferencias.
- ...

Yo puedo influir en parte de esos datos.. NO EN TODO. No es algo que yo controle directamente. Aunque si puedo INFLUIR en parte. Principalmente en:

- Instrucciones / límites que pongo al agente
- Información acerca de mi, mis preferencias
- Información de servidores MCP disponibles

Lo que pasa es que HAY QUE TENER MUCHO CUIDADO CON EL CONTEXTO ya que: Cada modelo es capaz de manejar un determinado tamaño de contexto. Hay modelos que pueden recibir 1M de tokens de contexto. Un token, en palabras es más o menos una palabra. Pero una imagen de 4Mbs ocupa muchos más tokens que una palabra. Lo que se mandan en este caso no son palabras, sino bytes. Otros modelos solo les puedo pasar 200k tokens de contexto.

Ese contexto se va llenando. La herramienta de chat lo que suele hacer es ir ANEXANDO cada nuevo mensaje y cada nueva respuesta , cada salida de un programa que se ejecute al CONTEXTO ANTERIOR, para dar lugar al CONTEXTO SIGUIENTE. Pero llega un momento que se llena... no lo lleno yo normalmente.. no escribo tanto. Lo llena:

- Las respuestas del modelo (que esos si enrollan mucho)
- Las respuestas/salidas de los programas que la IA ejecuta.

Qué pasa cuando se llena.. Hay que RESUMIRLO (compactación del contexto) para que el modelo pueda seguir recibiendo nuevo contexto sin perder información importante. Hay chats que te permite forzadamente hacer una compactación.. otro no... la hacen ellos. Normalmente no queremos que se haga en manual... El problema es que se resume.. y al resumir se PIERDE información.

- Se pierde mucho del historial de la conversación.
- De lo que yo he dicho
- De lo que ella ha dicho
- Y entonces el agente ya no va a hacer lo mismo o al menos no de la misma manera.
- Y ESTO ES UN PROBLEMA

Cuando tenemos un contexto muy grande también es problemático:

- En general los modelos recuerdan menos / le dan menos importancia a lo que hay por el medio del contexto.. y le dan más importancia a lo primero y lo último. Si lo pensaís es un poco lo que nos pasa a los humanos. Recordamos mejor el principio y el final de una historia, pero los detalles intermedios se nos escapan con facilidad.
- El otro problema es el COSTE de la petición. Los modelos cobran por petición.. y por tokens. Cuanto más envíe, más me cobran.

Hay que sopesar mucho, si me interesa compactar una conversación o no. Teniendo en cuenta eso.. y teniendo en cuenta que SE PIERDE MUCHA INFORMACIÓN.

Por eso, nos interesa mucho usar archivos que inyecten al contexto cosas que SIEMPRE QUIERO QUE ESTEN AHI.

En la compactación lo que se resume es el historial de la conversación, incluyendo tanto lo que yo he dicho como lo que el agente ha respondido, para reducir el tamaño del contexto y permitir que el modelo siga recibiendo nueva información sin perder lo esencial.

Pero vuelve a entrar desde el principio:

- Instrucciones / límites que ponemos al agente (aque se crean mediante archivos de texto)
- Información del entorno y de las herramienats que el modelo tiene disponibles
- Información de servidores MCP disponibles.

- Información acerca de mi, mis preferencias.
- ...

Lo que ponga en esa parte del contexto (muchas veces se inyecta desde ficheros de texto) siempre está. NUNCA SE PIERDE... por eso es tan importante.

Cuando uso una de estas herramienats, la herramienta va aprendiendo de cómo yo quiero que se hagan las cosas... esa información está en el historial de la conversación, y si se compacta, puede perderse.

Por eso el COMO quiero que se hagan las cosas suelo ponerlo en ficheros que se inyecten SIEMPRE al contexto.

PROBLEMA... Que yo no soy una máquina. Yo soy humano.. Y muchas veces ni yo sé lo que quiero y menos cómo lo quiero. Y lo voy descubriendo en paralelo mientras hablo con la IA.

Por suerte le puedo decir al agente de vez en cuando... Oye, todo lo que has aprendido llevalo a un fichero XXXXX.md para así que quede registrado y no se pierda cuando se compacte el historial.

Esto es a nivel de chat. Por eso es muy importante abrir muchos chats.Si pongo todo lo que se me ocurre en un único chat... genero un contexto muy heterogéneo y difícil de manejar, y además corro el riesgo de perder información importante cuando se compacte el historial.

Lo que me interesa es que el contexto tenga información de un tipo de tarea/proyecto/tema.

Los AGENTES van generando una MEMORIA para perder lo menos posible cuando se haga compactación. Los datos relevantes de cosas que han ocurrido son guardadas en ficheros de texto o en BBDD por las IAs.. y pueden consultarlas más adelante si lo necesitan.

ARCHIVOS IMPORTANTES con los que influir a nuestros agentes de IA que vamos a tener en un proyecto.

No todas las herramientas me permiten configurar esto. Y algunas me permiten hacerlo a traés de la propia herramienta, pero no mediante los archivos directamente. Lo ideal es usar una herramienta que permita configurar y mantener archivos de contexto de manera directa, asegurando que la información relevante siempre esté disponible para los agentes de IA sin depender únicamente del historial de la conversación.

Una herramienta como VSCODE ES IDEAL para este propósito, ya que permite gestionar archivos de contexto de manera directa y mantener la información relevante siempre disponible para los agentes de IA.

En VSCode abriremos una carpeta.. para un proyecto en el que quiera trabajar:

- Desarrollo de software
- Proyecto de gestión de lo que sea
- Puede ser un análisis financiero que quiero hacer sobre unas cuentas...

En esa carpeta crearé un directorio llamado .github Dentro de esa carpeta podré poner archivos y más carpetas.. que se usarán para fijar/influir en el contexto de los agentes de IA que trabajen en ese proyecto.

Todos los archivos que YO voy a editar en esa carpeta A PRIORI van a ser archivos con extensión .md

Pero esa extensión no es algo propio de las IAs. Es un formato llamado MARKDOWN que sirve para escribir documentación.

ES UN FORMATO ESTANDAR PARA DOCUMENTACIÓN, Y LAS IAs PUEDEN LEERLO Y ENTENDERLO SIN PROBLEMAS.